Universidad Autónoma de Madrid

Escuela Politécnica Superior

25 | 26

Proyecto APAUT-III

DIFFUSION MODELS



Miguel Borregón Sánchez

Belisa Yu

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C\Francisco Tomás y Valiente nº 11

img/campusEx

www.uam.es

UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR



Grado en Grado en Ciencia e Ingeniería de Datos

PROYECTO APAUT-III

DIFFUSION MODELS

Autor: Miguel Borregón Sánchez
Belisa Yu

May 2026

Algunos derechos reservados.

Este trabajo está bajo licencia Creative Commons

http://creativecommons.org/licenses/by-nc-sa/3.0/

Esta obra se puede copiar, distribuir y comunicar públicamente la obra así como crear obras derivadas bajo las siguientes condiciones:

- Debe reconocer los créditos manteniendo la autoría original y añadiendo la autoría de las modificaciones indicando de forma expresa y bien visible que el autor original no manifiesta ningún tipo de apoyo a as modificaciones realizadas así como al uso que se da de esta obra.
- No se puede utilizar esta obra con fines comerciales.
- Las modificaciones o ediciones de esta obra deben compartirse bajo una licencia idéntica a esta.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual *(arts. 270 y sgts. del Código Penal)*.

DERECHOS RESERVADOS
© Friday 1 May, 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

Miguel Borregón Sánchez
Belisa Yu
DIFFUSION MODELS

Miguel Borregón Sánchez
Belisa Yu
C\ Francisco Tomás y Valiente Nº 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

Resumen

Este trabajo aborda la implementación y comparativa de diferentes configuraciones de modelos de difusión para la generación de dígitos manuscritos del dataset MNIST. Se implementan los procesos de difusión Variance Exploding (VE) y Variance Preserving (VP), los samplers de Euler-Maruyama y Probability Flow ODE (PF-ODE), así como los schedules de ruido lineal, coseno y constante. La calidad de las imágenes generadas se evalúa mediante la métrica Bits per dimension (BPD). Los resultados muestran que PF-ODE supera a Euler-Maruyama en eficiencia y calidad al no introducir ruido durante el muestreo. Para el dataset completo de MNIST, VE ofrece mejores resultados debido a su mayor capacidad de exploración multimodal, mientras que para un solo dígito VP con schedule coseno es superior. Se recomienda el uso de PF-ODE con VP y schedule coseno, entrenando durante al menos 200 épocas, para obtener los mejores resultados.

Palabras clave

ÍNDICE

1 Introducción **1**

2 Estado del arte **2**

2.1 Fundamentos comunes para todos los modelos . . . 2

2.1.1 Ecuación diferencial estocástica (SDE) general . . . 2

2.1.2 Función de coste . . . 2

2.1.3 Fokker-Planck . . . 3

2.2 Procesos de difusión para inyección de ruido . . . 3

2.2.1 Variance Exploding (VE) / Brownian motion . . . 3

2.2.2 Variance Preserving (VP) / Ornstein-Unlenbeck . . . 3

2.3 Noise schedules . . . 4

2.3.1 Linear schedule . . . 4

2.3.2 Cosine schedule . . . 4

2.4 Arquitectura de red . . . 5

2.4.1 U-Net como backbone . . . 5

2.4.2 Condicionamiento temporal (Random Fourier Features/Positional encoding) . . . 5

2.5 Samplers para generación de imágenes . . . 6

2.5.1 Euler-Maruyama . . . 6

2.5.2 Predictor-Corrector . . . 6

2.5.3 Probability Flow ODE . . . 6

2.6 Métricas de calidad . . . 7

2.6.1 Bits per dimension (BPD) . . . 7

2.6.2 Fréchet Inception Distance (FID) . . . 7

2.6.3 Inception Score (IS) . . . 8

2.7 Generación controlada (Controllable generation) . . . 8

2.7.1 Class-conditional sampling . . . 8

2.7.2 Imputation . . . 9

3 Desarrollo de Software **10**

3.1 Análisis de requisitos . . . 10

3.1.1 Requisitos funcionales . . . 10

3.1.2 Requisitos no funcionales . . . 11

3.1.3 Casos de uso . . . 11

3.2 Diagrama de Gantt . . . 12

3.3 Validación y pruebas . . . 12

3.4 Desarrollo y garantía de calidad de software . . . 13

4 Resultados **14**
4.1 Resultados y comentarios 14
4.1.1 PF-ODE 15
4.1.2 EULER-MARUYAMA 17
5 Conclusiones **20**
Bibliografía **21**
Apéndices **23**
A Material técnico (derivaciones y fotos) **24**
A.1 Fokker-Planck. Ecuación completa 24
A.2 Movimiento Browniano 24
A.3 Variance Exploding (VE)/Brownian Motion. Derivación de g(t) 24
A.4 Variance Preserving (VP) / Ornstein-Unlenbeck. Distribución condicional 25
A.5 Comparativa entre VE y VP 25
A.6 Implicaciones de U-NET 26
A.7 Comparativa entre samplers 26
A.8 Construcción de Predictor-corrector 26
A.9 Comparativa de métricas y escenarios de uso 26
A.10 Impactos de Linear y Cosine 27
A.11 Derivaciones de mu_t y $sigma_t$ 27
A.11.1 VE (Variance Exploding) 27
A.11.2 VP (Variance Perserving) 28
A.12 Ilustraciones de los resultados 29
A.12.1 Ilustraciones para PF-ODE 29
A.12.2 Ilustraciones para EULER-MARUYAMA 33
B Anexo **38**
B.1 Aproximación de Hutchison 38
B.2 Square root schedule 38

1

Introducción

Los modelos generativos han revolucionado el campo de la inteligencia artificial en los últimos años, permitiendo crear contenido realista como imágenes, texto o audio. Entre ellos, los modelos de difusión han aparecido como una de las técnicas más prometedoras, superando en calidad a las GANs en numerosas tareas. Su capacidad para generar imágenes de alta resolución y su fundamentación matemática los han convertido en el estado del arte actual.

Este trabajo aborda la generación de imágenes mediante modelos de difusión, una de las técnicas más avanzadas en inteligencia artificial generativa. El objetivo es comparar diferentes configuraciones —procesos VE y VP, samplers (Euler-Maruyama y PF-ODE) y schedules de ruido (lineal, coseno y constante)— sobre el dataset MNIST. Se evalúa la calidad mediante la métrica BPD y se analiza la influencia de los hiperparámetros en la calidad y diversidad de las muestras. El documento se organiza en cinco capítulos: estado del arte, desarrollo software, resultados, conclusiones y bibliografía.

2

Estado del arte

2.1 Fundamentos comunes para todos los modelos

2.1.1 Ecuación diferencial estocástica (SDE) general

La ecuación diferencial estocástica (SDE) general que define un proceso de difusión en el sentido de Itô es:

$$dx = f(x,t)dt + g(t)dw$$

donde:

- f(x,t) es el coeficiente de deriva (drift)
- g(t) es el coeficiente de difusión
- dw es un proceso de Wiener

Esta ecuación modela la evolución temporal de x(t), donde el primer componente *f(x,t)dt* representa la componente determinista (deriva), mientras que el segundo término g(t)dw representa la componente estocástica (difusión).

2.1.2 Función de coste

La función de coste es un modelo utilizado para aprender durante el entrenamiento. Para nuestro caso, diffusion models, la función de coste viene dada por:

$$\min_{\theta} \mathbb{E}_{t\sim\mathcal{U}[0,T]} \left[\lambda(t)\, \mathbb{E}_{\mathbf{x}(0)\sim p_{\text{data}}(\mathbf{x})}\, \mathbb{E}_{\mathbf{x}(t)\sim p_{0t}(\mathbf{x}(t)|\mathbf{x}(0))} \left[\|\mathbf{s}_\theta(\mathbf{x}(t),t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0))\|^2\right]\right]$$

Sin embargo, al usar ruido gaussiano, la ecuación se simplifica obteniendo:

$$\min_{\theta} \mathbb{E}_{t,\mathbf{x}_0,\mathbf{z}} \left[\lambda(t) \left\|\mathbf{s}_\theta(\mathbf{x}_0 + \sigma_t \mathbf{z}, t) + \frac{\mathbf{z}}{\sigma_t}\right\|^2\right]$$

Esto es debido a que cuando añadimos ruido gaussiano, la distribución es: $x_t = x_0 + \sigma_t z$ donde z N(0,I), lo

que implica que $x_t|x_0\ N(x_0, \sigma_t^2 I)$

Para una distribución normal, el logaritmo de la densidad es: $log\ p(x) = -\frac{(x-mu)^2}{2\sigma^2} + constante$

Por tanto: $\nabla_{x_t} log\ p_0 t(x_t|x_0) = -\frac{\sigma_t z}{\sigma_t^2} = -\frac{z}{\sigma_t}$

Sustituyendo en la fórmula general: $\left\|\mathbf{s}_\theta(\mathbf{x}_t, t) - \left(-\frac{\mathbf{z}}{\sigma_t}\right)\right\|^2 = \left\|\mathbf{s}_\theta(\mathbf{x}_t, t) + \frac{\mathbf{z}}{\sigma_t}\right\|^2$

2.1.3 Fokker-Planck

Ecuación diferencial parcial que describe la evolución temporal de la función de densidad de probabilidad $p(x, t)$ de un proceso estocástico. Para un proceso de Itô en una dimensión espacial x, definido por la SDE:

$$dX_t = \mu(X_t, t)\, dt + \sigma(X_t, t)\, dW_t$$

donde W_t es un proceso Wiener estándar, $\mu(X_t, t)$ es el coeficiente de deriva y $\sigma(X_t, t)$ es el coeficiente de difusión.

La formulación matemática completa se incluye en el Apéndice.

2.2 Procesos de difusión para inyección de ruido

2.2.1 Variance Exploding (VE) / Brownian motion

Los procesos de difusión conocidos como Variance Exploding (VE) son un tipo de ecuación diferencial estocástica utilizados en los modelos generativos basados en score. Su funcionamiento consiste en añadir ruido progresivamente a los datos originales, de forma que la varianza de los datos perturbados aumenta (explota) con el tiempo hasta convertirse en una distribución gaussiana con varianza muy grande ——potencialmente hasta infinito—— en intervalos finitos o infinitos.

Se define como una Ecuación Diferencial Estocástica en el sentido de Itô (Itô SDE) de la forma $dx = g(t)dw$, donde w es un movimiento browniano estándar. Como el coeficiente de deriva es nulo, es decir, f(x,t) = 0, la SDE se simplifica y obtenemos:

$$dx = g(t)dw$$

Tras su derivación (*Ver: Apéndice VE/Brownian motion. Derivación de g(t)*), obtenemos que $g(t) = \sigma(t)\sqrt{2 \ln \frac{\sigma_{max}}{\sigma_{min}}}$

2.2.2 Variance Preserving (VP) / Ornstein-Unlenbeck

Los procesos de difusión conocidos como Variance Preserving (VP) son un tipo de ecuación diferencial estocástica en los que el ruido inyectado en cada paso de ruido hacia adelante se equilibra exactamente, de manera que la

varianza marginal total del sistema en evolución se mantiene constante, normalmente normalizada a la unidad. Lo que permite un entrenamiento estable y una dinámica de muestreo controlada.

Se define como una Ecuación Diferencial Estocástica en el sentido de Itô:

$$d\mathbf{x} = -\frac{1}{2}\mathbf{x}\,dt + d\mathbf{w}$$

Resolviendo la ecuación obtenemos: $x_t = x_0 e^{-t/2} + e^{-t/2}\int_0^t e^{s/2} dw_s$

Distribución condicional, ver en Apéndice, Material técnico, VP/Ornstein-Unlenbeck. Distribución Condicional

Adicionalmente se presenta una comparativa entre VE y VP en Apéndice, Material técnico. Comparativa entre VE y VP

2.3 Noise schedules

Definición: forma en que se define cómo crece el ruido a lo largo del tiempo en un proceso de difusión

2.3.1 Linear schedule

Linear schedule define la varianza del ruido como una función lineal del tiempo:

$$\beta_t = \beta_{\min} + \frac{t}{T}(\beta_{\max} - \beta_{\min})$$

Aplica ruido constante durante todo el espacio temporal, lo que lo hace fácil de implementar e interpretable. Sin embargo puede provocar cambios bruscos en las etapas finales de la difusión, donde niveles altos de ruido pueden sobrepasar los detalles sutiles de los datos. En generación de imágenes, puede dar a resultados borrosos o con artefactos, ya que el modelo tiene dificultades para refinar características de grano fino cuando el ruido aumenta demasiado rápido.

2.3.2 Cosine schedule

Cosine schedule se define mediante la siguiente expresión:

$$\bar{\alpha}(t) = \cos^2\left(\frac{t+s}{1+s} \cdot \frac{\pi}{2}\right)$$

Consiste en ralentizar el crecimiento del ruido al inicio y al final del proceso de difusión, mientras que lo acelera en la parte media.

Se trata de una distribución no lineal del ruido, que evita que los niveles altos de ruido destruyan los detalles sutiles de los datos en las etapas inciales. Además, al dedicar más pasos a niveles de ruido intermedios, permite

que el modelo aprenda mejor las características de grano fino. Como resultado, en imágenes, se obtienen resultados más nítidos y con menos artefactos.

2.4 Arquitectura de red

2.4.1 U-Net como backbone

La U-net se trata de una red convolucional desarrollada para la segmentación de imágenes biomédicas. Se basa en una red neuronal totalmente convolucional cuya arquitectura se modificó y amplió para trabajar con menos imágenes de entrenamiento y producir una segmentación más precisa.

Consiste en analizar la imagen de forma global para capturar el contexto, y luego ir evaluando sus características a diferentes escalas (dividiendo la imagen en 'trozos'), para luego recomponerla y obtener el resultado esperado Inputs:

- imagen con ruido
- instante de tiempo t (positional encoding)

Output: Score $\nabla \, log \, p_t$ -> tensor de la misma dimensión que la entrada

Implicaciones, ver en Apéndice

2.4.2 Condicionamiento temporal (Random Fourier Features/Positional encoding)

El tiempo t es un escalar que debe ser introducido en la red para que esta sepa en qué nivel de ruido se encuentra. Para ello, se hace uso de una codificación que transforma el escalar en un vector de alta dimensión.

Random fourier features permite que una red neuronal simple imite el comportamiento de métodos kernel más sofistificados cuando hay suficientes neuronas; convierte un simple número (el tiempo t) en un montón de números diferentes que ayudan a la red a entender mejor ese tiempo. Con fórmula original:

$$z(x) := \frac{1}{\sqrt{D}}[\cos\langle\omega_1, x\rangle, \sin\langle\omega_1, x\rangle, \ldots, \cos\langle\omega_D, x\rangle, \sin\langle\omega_D, x\rangle]^T$$

En los casos estudiados consideraremos que x = t (escalar), por lo que la fórmula se puede simplificar a:

$$z(t) := \frac{1}{\sqrt{D}}[\cos(2\pi\omega_1 t), \sin(2\pi\omega_1 t), \ldots, \cos(2\pi\omega_D t), \sin(2\pi\omega_D t)]^T$$

Donde las frecuencias ω_k se muestrean de una distribución y se fijan al inicio del entrenamiento. Lo que permite capturar múltiples escalas de variación.

2.5 Samplers para generación de imágenes

Se presenta una comparativa de los samplers en Apéndice, Material técnico. Comparativa samplers

2.5.1 Euler-Maruyama

Se trata de un método simple para discretizar y resolver una SDE. En los diffusion models es usado para la generación de imágenes avanzando paso a paso desde ruido puro hasta una imagen limpia, resolviendo la SDE reversa. Discretización:

- Dividir el intervalo [0,T] en N subintervalos iguales de ancho $\Delta t > 0$:

$$0 = \tau_0 < \tau_1 < ... < \tau_N = Ty\Delta t = T/N$$

- Establecer $Y_0 = x_0$
- Definir recursivamente Y$_n para 0 \leq n \leq N-1$ mediante $Y_{n+1} = Y_n + \alpha(Y_n, \tau_n)\Delta t + b(Y_n, \tau_n)\Delta W_n$, donde $\Delta W_n = W_{\tau_{n+1}} - W_{\tau_n}$

En el contexto de los diffusion models, esta fórmula se adapta a la SDE revsersa, incorporando el término del score $\mathbf{s}_\theta(\mathbf{x}_t, t)$.

2.5.2 Predictor-Corrector

Los métodos predictor-corrector pertecenen a una clase de algoritmos diseñados para integrar ecuaciones diferenciales ordinarias, es decir, para encontrar una función desconocida que satisfaga una ecuación diferencial dada. Consta de dos pasos:

- Predicción: parte de una función ajustada a los valores de la función y la derivada en un conjunto de puntos precedentes para extrapolar el valor de esta función en un punto nuevo, es decir, da una primera aproximación de cuál será el siguiente valor.
- Corrección: refina la aproximación inicial usando el valor previsto de la función y otro método (Langevin) para interpolar el valor de esa función desconocida en el mismo punto subsiguiente. Lo que permite combinar información pasada y presente, ajustando el valor predicho para que sea más coherente con la ecuación diferencial.

Construcción del modelo en Apéndice

2.5.3 Probability Flow ODE

Se trata de una forma alternativa de generar imágenes que no usa ruido durante el proceso de muestreo, a diferencia de otros métodos como Euler-Maruyama o Predictor-Corrector. Convierte el problema estocástico (con aletoriedad) en un problema determinista (sin aletoriedad), lo que lo hace más rápido y estable, aunque potencialmente menos diverso. Con fórmula matemática:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{f}(\mathbf{x}_t, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log q_t(\mathbf{x}_t)$$

donde:

- $\mathbf{f}(\mathbf{x}_t, t)$ es el coeficiente de deriva
- $g(t)$ es el coeficiente de difusión
- $\nabla_{\mathbf{x}} \log q_t(\mathbf{x}_t)$ es el score estimado por la red neuronal

2.6 Métricas de calidad

Se presenta una comparación de las métricas en: Apéndice, Material técnico. Comparativa de métricas y escenarios de uso

2.6.1 Bits per dimension (BPD)

Métrica comunmente usada para evaluar modelos generativos, especialmente en el ámbito de procesamiento de imágenes y sonido. Mide la cantidad de información, en bits, que necesita el modelo para reconstruir cada dimensión de los datos.

Generalmente se calcula a partir de la pérdida de log-verosimilitud negativa (NNL) o la divergencia KL que produce el modelo. Luego, el resultado se promedia y se divide por $log(2)$ para convertir las unidades de nats a bits, lo que facilita la comparación entre diferentes modelos.

$$BPD = \frac{NLL}{D \cdot ln(2)}$$

Donde

- D es el número total de dimensiones de la imagen (M x N x C)
- NNL es la log-verosimilitud negativa. Mide cuán de probable es que el modelo genere una imagen real: $NLL = -log\ p(x)$, donde p(x) es la probabilidad que el modelo asigna a la imagen x

2.6.2 Fréchet Inception Distance (FID)

Métrica usada para determinar la calidad de las imágenes creadas por un modelo generativo, comparando la distribución de las imágenes generadas con la distribución de un conjunto de imágenes de verdad, para ello mide la distancia entre dos distribuciones de probabilidad en el espacio de características de una red neuronal preentrenada (Inception v3).

El FID asume que las distribuciones son Gaussianas, lo que permite calcular la distancia de Fréchet mediante la siguiente expresión:

$$d_F(\mathcal{N}(\mu, \Sigma), \mathcal{N}(\mu', \Sigma'))^2 = \|\mu - \mu'\|_2^2 + \operatorname{tr}\left(\Sigma + \Sigma' - 2\left(\Sigma^{\frac{1}{2}} \cdot \Sigma' \cdot \Sigma^{\frac{1}{2}}\right)^{\frac{1}{2}}\right)$$

Un valor bajo FID indica que las imágenes generadas son de alta calidad y diversas, ya que su distribución es similar a la de las imágenes reales. Por el contrario, un valor alto sugiere diferencias signifcativas, con imágenes borrosas, con artefactos o poca diversidad. Sin embargo, una limitación encontrada, es que se requiere un número considerable de imágenes para estimar correctamente las covarianzas y asume que las distribuciones son gaussianas. Además, según demostraron $Chong\ y\ Forsyth$, FID está sesgado estadísticamente, es decir, su valor esperado sobre datos finitos no es su valor real.

2.6.3 Inception Score (IS)

Métrica usada para evaluar la calidad de las imágenes creadas por un modelo generativo de imágenes. El IS se calcula en función de la salida de un modelo de clasificación de imágenes Inception v3 preentrenado e independiente. Se maximiza cuando cumple las siguientes condiciones:

- Se minimiza la entropía de la distribución de etiquetas predicha por el modelo Inception v3 para las imágenes generadas. Es decir, predice con confianza una única etiqueta para cada imagen
- Las predicciones del modelo de clasificación se distribuyen uniformemente entre todas las etiquetas posibles.

$$IS(p_{gen}, p_{dis}) := \exp\left(\mathbb{E}_{x\sim p_{gen}}\left[D_{KL}\left(p_{dis}(\cdot|x)\,\middle|\,\int p_{dis}(\cdot|x)p_{gen}(x)dx\right)\right]\right)$$

donde:

- p_{gen} es la distribución de las imágenes generadas
- $p_{dis}(\cdot|x)$ es la distribución de etiquetas predichas por Inception v3 para la imagen x
- D_{KL} es la divergencia de Kullback-Leibler

Un valor alto indica que las imágenes generadas son nítidas y diversas. Sin embargo, no compara con imágenes reales, por lo que hoy en día es complementada con FID.

2.7 Generación controlada (Controllable generation)

2.7.1 Class-conditional sampling

Técnica poderosa para la generación de datos sintéticos que permite producir muestras de datos adaptadas a condiciones o valores de características específicos. Para ello, en los diffusion models, se añade información de la clase al proceso, mediante dos formas principales:

- Se introduce la clase y como una entrada adicional a la red neuronal

$$\mathbf{s}_\theta(\mathbf{x}_t, t, y)$$

- La clase y se convierte en un vector y se inyecta en la red

Durante el entrenamiento, la red aprende a asociar cada clase con sus características distintivas. En la generación, se especifica la clase deseada y el modelo produce imágenes pertenecientes a esa categoría

2.7.2 Imputation

Técnica estadística que consiste en reemplazar los datos faltantes con valores sustituidos. En diffusion models, consiste en rellenar partes de una imagen que faltan o están ocultas, de forma coherente.

En diffusion models, una técnica común es *RePaint*, que consiste en modificar el proceso de muestreo para forzar que la región visible se mantenga fiel a la imagen original mientras se genera el contenido de la región oculta. Mediante el siguiente proceso iterativo:

$$x_{t-1}^{\text{known}} \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t)\mathbf{I}) \quad \text{(known part)}$$

$$x_{t-1}^{\text{unknown}} \sim \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad \text{(generated part)}$$

$$x_{t-1} = m \odot x_{t-1}^{\text{known}} + (1-m) \odot x_{t-1}^{\text{unknown}} \quad \text{(combine)}$$

donde m es una máscara binaria. La parte conocida se mantiene fiel a la imagen original, mientras que la parte oculta se genera con el modelo de difusión.

Explicación del modelo: para los píxeles conocidos, no los generamos sino que los 'difuminamos' según el nivel de ruido en el paso t-1. Es decir, matenemos la fidelidad de la imagen original. Para los píxeles desconocidos, los generamos con el modelo de difusión estándar. Y por último, mezclamos las dos partes para formar la imagen completa en el paso t-1.

3

Desarrollo de Software

En esta sección se desarrollará una descripción de los aspectos prácticos relacionados con la implementación del sistema generativo basado en modelos de difusión. Se presentan los requisitos funcionales y no funcionales, un digrama de Gantt donde se mostrará la evolución temporal, las estrategias de validación y pruebas, así como las prácticas de calidad adoptadas.

3.1 Análisis de requisitos

3.1.1 Requisitos funcionales

ID	Requisito	Descripción
RF1	Cargar imágenes	El sistema debe cargar el dataset MNIST de la librería torchvision.dataset y preprocesar las imágenes
RF2	Entrenar subconjuntos	El sistema debe permitir entrenar un subconjunto de los datos
RF3	Proceso de difusión	El sistema debe implementar un proceso de difusión de tipo Variance Preserving (VP) y un proceso de difusión de tipo Variance Exploding (VE)
RF4	Euler-Maruyama	El sistema debe implementar el sampler de Euler-Maruyama
RF5	Probability Flow - ODE	El sistema debe implementar el sampler de Probability Flow - ODE
RF6	Noise schedules	El sistema debe implementar tres noise schedules diferentes (Linear Noise Schedule, Cosine Noise Schedule, uno a elegir) y permitir su elección
RF7	Log-likelihood	El sistema debe calcular la verosimilitud de una imagen a través de la función compute ODE
RF8	Métricas de calidad	El sistema debe calcular la métrica de calidad BPD
RF9	Visualización de resultados	El sistema debe visualizar la evolución de las imágenes generadas

Tabla 3.1: Tabla 1: Requisitos funcionales del sistema

En el cuadro 3.1 se presentan los requisitos funcionalies del sistema.

3.1.2 Requisitos no funcionales

En el apartado 3.1.2 se presentan los requisitos no funcionales del sistema.

ID	Requisito	Descripción
RNF1	Entorno de desarrollo	Python 3.10 o superior
RNF2	Hardware	CPU para entrenamiento de subsets pequeños, GPUU para entrenamiento del dataset completo
RNF3	Tiempo de entrenamiento	Subset pequeño con < 30 epochs, 10-15 min en CPU; Dataset completo > 100 epochs, 1hora en GPU
RNF4	Tiempo de generación	Subset pequeño 10-15 segundos en CPU; Dataset completo 20-30 segundos en CPU

Tabla 2: Requisitos no funcionales del sistema

3.1.3 Casos de uso

CU-01: Entrenar modelo

Este caso de uso describe el proceso mediante el cual un investigador entrena un modelo de difusión utilizando el dataset MNIST. Como precondición, es necesario disponer previamente del conjunto de datos descargado. El proceso comienza cuando el usuario ejecuta el script de entrenamiento y configura los principales hiperparámetros del modelo, incluyendo el número de épocas (*epochs*), el tamaño de lote (*batch_size*), la tasa de aprendizaje (*learning_rate*) y el número de pasos de difusión (*n_steps*). A continuación, el sistema se encarga de cargar las imágenes. Durante el entrenamiento, el modelo se optimiza mostrando la evolución de la función de pérdida en cada época. Finalmente, una vez completado el proceso, el sistema guarda los pesos del modelo en el archivo `check_point.pth`, dejando el modelo listo para su uso en tareas de generación de imágenes. Este caso de uso está relacionado con los requisitos funcionales RF1 y RF2.

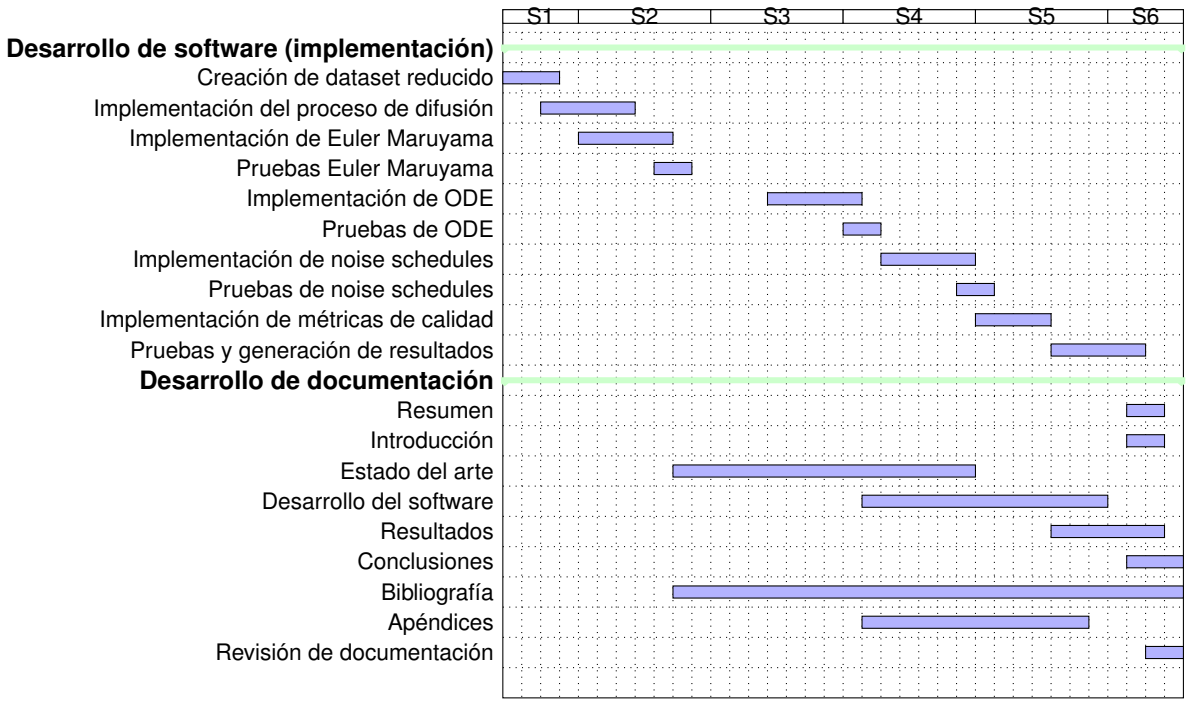
CU-02: Generar imágenes

Este caso de uso describe el proceso mediante el cual un investigador genera nuevas imágenes a partir de ruido utilizando un modelo de difusión previamente entrenado. Como precondición, es necesario disponer de un modelo ya entrenado. El proceso comienza cuando el usuario ejecuta el script de generación y selecciona el método de muestreo a utilizar, pudiendo elegir entre Euler-Maruyama o Probability Flow ODE (PF-ODE). A continuación, el usuario define el tipo de proceso de difusión, ya sea Variance Exploding (VE) o Variance Preserving (VP), así como el *noise schedule* que determina la evolución del ruido a lo largo del tiempo. Posteriormente, el sistema genera una muestra inicial de ruido puro y aplica de forma iterativa el sampler seleccionado para ir eliminando progresivamente el ruido. Finalmente, el sistema muestra las imágenes generadas como resultado del proceso. Este caso de uso está relacionado con los requisitos funcionales RF3, RF4, RF5, RF6 y RF9.

CU-03: Comparar schedules

Este caso de uso describe el proceso mediante el cual un investigador compara el comportamiento de distintos *noise schedules*, concretamente los enfoques linear y cosine, en el contexto de modelos de difusión. Como precondición, es necesario haber entrenado el modelo al menos una vez con cada uno de los dos schedules. El proceso comienza cuando el usuario selecciona el tipo de schedule y configura los parámetros asociados al proceso de difusión. A continuación, el sistema entrena el modelo utilizando el schedule elegido. Este procedimiento se repite para ambos enfoques, permitiendo obtener modelos entrenados bajo diferentes configuraciones de ruido. Finalmente, el sistema genera imágenes con cada uno de los modelos y las presenta al usuario, facilitando su evaluación y comparación visual. Como resultado, el investigador puede analizar el impacto de cada schedule en la calidad de las imágenes generadas. Este caso de uso está relacionado con los requisitos funcionales RF6 y RF9.

3.2 Diagrama de Gantt



3.3 Validación y pruebas

Se han realizado pruebas unitarias para verificar el correcto funcionamiento de cada componente del sistema:

1.– **Carga de datos:** Se verifica que el dataset MNIST se carga correctamente

2.– **Selección de subsets**: Se verifica que es posible extraer un subset del dataset (imágenes de un/unos número/s concretos) para su entrenamiento

3.– **Proceso de difusión**: Ser verifica que μ_t y σ_t producen tensores con las dimensiones correctas

Para validar el correcto funcionamiento del sistema de forma temprana y con recursos limitados, se han realizado pruebas utilizando un subconjunto del reducido del dataset MNIST (se ha probado con el dígito 3, y los dígitos 3 y 8). Esto permitió:

1.– Reducir el tiempo de entrenamiento

2.– Permitir el uso de CPU en lugar de GPU (para sistemas que no disponen de gpu)

3.– Detectar errores en las funciones del proceso de difusión

4.– Verificar que los samplers generan imágenes con la estructura esperada

3.4 Desarrollo y garantía de calidad de software

Para garantizar la calidad y reproducibilidad del software desarrollado, se han seguido una serie de pasos durante el ciclo de vida del proyecto.

Se ha hecho uso de la plataforma de GitHub para el control de versiones y la integración continua del proyecto. Lo que ha permitido gestionar las diferentes versiones de código, así como facilitar la colaboración entre los miembros del equipo de trabajo.

El repositorio incluye un historial completo de commits, lo que permite:

- Rastrear la evolución del proyecto.
- Revertir cambios en caso de errores.
- Colaborar de forma simultánea sin conflictos.

4

Resultados

Para efectos de la práctica, se han abordado diversas pruebas para verificar el correcto funcionamiento de los samplers y diffusion models antes del entrenamiento con el dataset completo. Para ello se han hecho pruebas, con sus correspondientes resultados, de:

- Euler-Maruyama con VE y ruidos raíz cuadrada (squared schedule) y exponencial (exponential schedule)
- Euler-Maruyama con VP y ruidos constante, linear y cosine
- PF-ODE con VE y ruido de raíz cuadrada (squared schedule) y exponencial (exponential schedule)
- PF-ODE con VP y ruido constante, linear y cosine

Una vez completado las pruebas con subsets se han abordado los entrenamientos con el dataset completo para cada una de las situaciones. Todo ello se verá reflejado en cada subapartado

Antes de iniciar con la puesta en vista de los resultados, destacamos las diferentes derivaciones llevadas a cabo para los parámetros mu_t y $sigma_t$ de cada noise-schedule y sampler. *Ver en Apéndice, Material técnico, Derivaciones de* mu *y* $sigma$

4.1 Resultados y comentarios

Se han realizado unas pruebas iniciales siguiendo las siguientes características: Subset únicamente del dígito 3, uso de CPU (debido a las limitaciones de GPU), 30 épocas de entrenamiento con un learning rate de 1e-3, y 800 pasos, en el que para VE se han empleado pasos de menor tamaño (ultimos 100 pasos de 10 en 10), mientras que para VP pasos de 50 en 50. Cabe destacar que las elecciones de los parámetros se deben a diversas pruebas realizadas con menor número. Se hizo una evaluación con 10 épocas y 500 pasos, donde se presentaba mucho ruido e imágenes poco definidas.

Posteriormente, una vez evaluado el correcto funcionamiento, se prueba con el dataset completo, siguiendo las siguientes características: 100 épocas, 800 pasos para VP y 2000 para VE. Con el objetivo de que el sistema devuelva todos los números y no se produzca el colapso de modos (el modelo solo aprende a generar una o pocas variantes, ignorando el resto), se ha aplicado generación condicionada por clases (class-conditional generation), de manera que se proporciona al modelo la etiqueta (y) del dígito deseado (en nuestro caso 0-9) como una entrada durante el entrenamiento y la generación.

Para la evaluación de calidad se ha hecho uso de BPD, se podrá observar que todos los resultados son

buenos pero no excelentes, esto es debido principalmente al número de épocas con la que se han entrenado las redes.

Dado que las pruebas con VE raíz cuadrada son adicionales (no se piden explícitamente en la práctica), los resultados se presentarán en el apartado de Apéndice.

4.1.1 PF-ODE

Variance Exploding (VE) con inyección exponencial

Se han realizado las pruebas con inyección exponencial, primero usando un Subset con el dígito 3:

En la apéndice A.12.1 se muestra la predicción obtenida con un Subset únicamente del dígito 3, empleando PF-ODE, variance exploding (VE) con inyección exponencial.

Se observan unos resultados muy buenos. Esto se debe al crecimiento gradual del ruido, que permite primero un aprendizaje global y luego los detalles finos, mientras que ODE elimina el ruido durante el muestreo. Con un bpd de: mean = 6.6922 y desviación = 0.0021. A continuación, se muestra el resultado con el dataset entero:

En la apéndice A.12.2 se muestra la predicción obtenida para el dataset completo, empleando PF-ODE, variance exploding (VE) con inyección exponencial.

Observamos unos números que presentan bastante ruido y están poco definidos, esto se debe principalmente al número de épocas con el que se entrenó. Por último se representan las soluciones a color.

En la apéndice A.12.1 se muestra la predicción obtenida a color con el dataset completo, empleando PF-ODE, variance exploding (VE) con inyección exponencial.

Observamos que los números no se representan adecuadamente, mostrando mucho ruido además de no ser bien reconocibles. Esto se debe sobre todo al número de épocas con el que se ha probado.

Variance Perserving (VP) con inyección linear (*Linear Schedule*)

Se muestra el resultado obtenido con el Subset del dígito 3:

En la apéndice A.12.1 se muestra la predicción obtenida con un Subset únicamente del dígito 3, empleando PF-ODE, variance perserving (VP) con inyección linear schedule.

Como se ha podido comprobar, efectivamente los '3' están mejor definidos y presentan menos ruido. El bpd (bits per dimension) obtenido tiene media = 4.8912 y desviación = 0.0132, lo que implica que nuestro modelo se encuentra en un rango bueno, es decir, aprende correctamente la distribución y la calidad de las imágenes son aceptables. Sin embargo, para que sean muy buenos/excelentes se recomienda el aumento del número de épocas con las que se entrena, por ejemplo con 50 épocas. A continuación, se ha hecho la prueba con el dataset completo, los resultados esperados presentarán las mismas características que el dígito único.

En la apéndice A.12.1 se muestra la predicción obtenida con el dataset completo, empleando PF-ODE, variance perserving (VP) con inyección linear schedule.

Observamos que las imágenes generadas están relativamente bien definidas, cada número se puede identificar sin esfuerzo. Sin embargo, presentan cierto ruido, esto se debe al número de épocas con el que se ha entrenado el modelo y el número de pasos para su predicción, ya que con 10 dígitos la distribución es multimodal y más compleja requiriendo más recursos para aprender todos los modos correctamente. La imagen superior se generó con 100 épocas de entrenamiento y 800 pasos, se pueden diferenciar claramente los números y el ruido presentado es bajo, si se aumenta el número de épocas, por ejemplo 150, y el número de pasos, se podrán obtener resultados con menos ruido todavía, convergiendo a casi 0. A continuación se muestran los resultados a color, el BPD obtenido presenta media = 4.4148 y desviación = 5.2815:

En la apéndice A.12.1 se muestra la predicción obtenida a color con el dataset completo, empleando PF-ODE, variance perserving (VP) con inyección linear schedule.

Variance Perserving (VP) con inyección coseno (*Cosine Schedule*)

Según la teoria, explicada en *Estado del arte*, con el uso de cosine se deben obtener mejores resultados que linear, debido a su sistema de inyección de ruido, el cual ralentiza la adición al principio y al final, preservando mejor los detalles finos y evitando ruido innecesario cuando la señal ya es débil, al contrario que linear, que añade ruido a ritmo constante.

En la apéndice A.12.1 se muestra la predicción obtenida con un Subset únicamente del dígito 3, empleando PF-ODE, variance perserving (VP) con inyección cosine schedule.

Como se puede observar, los valores obtenidos son muy satisfactorios. Analizando su bpd, obtenemos media de bpd = 8.1780, y desviación = 0.0589. Observamos que el resultado es peor que el obtenido para linear, sin embargo, la teoría dice justo lo contrario. Esto es debido al número de épocas con la que se ha entrenado el modelo, Cosine preserva más la estructura al inicio y aprende más lento pero mejor a largo plazo, por lo que necesita más épocas para converger. Por el contrario, linear destruye la señal al inicio dando lugar a un aprendizaje más rápido.

A continuación se muestra el resultado entrenado con todo el dataset.

En la apéndice A.12.1 se muestra la predicción obtenida con el dataset completo, empleando PF-ODE, variance perserving (VP) con inyección cosine schedule.

Observamos que las imágenes generadas están relativamente bien definidas, obteniendo un resultado similar a Linear. En teoría debería tener mayor resolución, pero debido al número de épocas y pasos establecidos no llega a apreciarse, al aumentar los valores, las diferencias serán mayores, sobre todo a la hora de evaluación métrica. A continuación se muestran a color, el BPD obtenido presenta media = 7.4707 y desviación = 3.6692:

En la apéndice A.12.1 se muestra la predicción a color obtenida con el dataset completo, empleando PF-ODE, variance perserving (VP) con inyección cosine schedule.

Variance Perserving (VP) con $\beta = 1$

El resultado esperado ha de ser de menor calidad que el obtenido con los dos métodos superiores (linear y cosine), sobre todo cosine. Esto es debido a que al establecer beta fija se está inyectando ruido de manera uniforme lo que degrada demasiado rápido la estructura semántica de la imagen (bordes y formas) desde etapas tempranas, dejando poco margen para que el modelo aprenda a reconstruir los detalles finos durante el reverse process.

En la apéndice A.12.1 se muestra la predicción obtenida con un Subset únicamente del dígito 3, empleando PF-ODE, variance perserving (VP) con β constante.

Como es de observar, los valores obtenidos son reconocibles, sin embargo, hay algún dígito que no parece estar tan bien definido (justo lo esperado). Analizando su bpd, obtenemos: media = 1.7420 y desviación = 0.0045. Estos resultados muestran cierta contradicción con lo indicado anteriormente, pero el resultado obtenido es totalmente razonable y coherente, ya que con 30 épocas (pocas) el modelo de β constante ha sido capaz de converger rápidamente a un distribución cercana a la real, logrando un BPD excepcionalmente bajo (menor a 2), lo que indica una calidad de generación muy alta. Este comportamiento era predecible, pues el schedule con β constante ($\beta = 1$) presenta una tasa de destrucción de señal uniforme y moderada, permitiendo que el modelo aprenda de forma estable desde las primeras épocas. A diferencia del linear o cosine qeu requieren mayor entrenamiento para converger. A continuación, se prueba con el dataset completo de MNIST.

En la apéndice A.12.1 se muestra la predicción obtenida con el dataset completo, empleando PF-ODE, variance perserving (VP) con β constante.

Observamos que los valores obtenidos son reconocibles, pero presentan claramente más ruido que los obtenidos con cosine o linear. A continuación se representan los resultados a color, el BPD obtenido presenta media = 3.7800 y desviación = 1.2220.

En la apéndice A.12.1 se muestra la predicción a color obtenida con el dataset completo, empleando PF-ODE, variance perserving (VP) con β constante.

4.1.2 EULER-MARUYAMA

En esta sección se presentarán los resultados obtenidos con Euler-Maruyama, los valores resultantes serán peores que los anteriores (implementados con Probability-Flow ODE), debido a que Euler-Maruyama introduce ruido estocástico en cada paso de integración, lo que incrementa la varianza en las trayectorias de muestreo y puede generar imágenes menos nítidas (ver teoria en *Estado de Arte*). Sin embargo, para datasets con grandes volúmenes de datos (mayores que MNIST), tenderá a devolver mejor calidad gracias al muestro Predictor-Corrector usado internamente.

De igual manera que para ODE, se harán pruebas con un dígito y posteriormente con todo el dataset. De nuevo, los bpd obtenidos son relativamente buenos, pero no llegan a excelentes debido a la cantidad de épocas con la que se entrenaron los modelos.

Variance Exploding (VE) con inyección exponencial

Se muestra la solución obtenido con un Subset únicamente con el número 3. Se esperan resultados donde los bordes aparecen ligeramente borrosos y la imagen presentará cierto ruido. Sin embargo, los dígitos serán reconocibles a simple vista.

En la apéndice A.12.2 se muestra la predicción con un Subset únicamente del dígito 3, empleando EM, variance exploding (VE) con inyección exponencial.

Podemos observar que los resultados obtenidos no son muy favorables, esto es debido a la naturaleza de EM y VE, que inyectan ruido estocástico en cada paso y no tienen una 'ruta' definida, por lo que requiere de muchas más épocas para converger. A continuación, se muestran los resultados obtenidos del dataset completo, una vez más, no se esperan resultados muy buenos.

En la apéndice A.12.2 se muestra la predicción el dataset completo, empleando EM, variance exploding (VE) con inyección exponencial.

Como podemos observar, los resultados obtenidos, con 150 épocas parecen ser bastante favorables, presentando cierto ruido, y algún dígito mal definido, como el 7; esto es debido a que VE requiere muchas más épocas para converger. Sin embargo, a color los números obtenidos son muy malos y presentan mucho ruido; debido a que VE amplifica el ruido por canal, haciendo que incluso si la estructura del dígito es reconocible (en nuestro caso no lo son casi ninguno), los colores parezcan aleatorios. Se muestra a continuación:

En la apéndice A.12.2 se muestra la predicción el dataset completo a color, empleando EM, variance exploding (VE) con inyección exponencial.

Variance Perserving (VP) con inyección linear (*Linear Schedule*)

El resultado esperado, ha de ser una imagen donde el 3 sea claramente reconocido, con cierto nivel de ruido.

En la apéndice A.12.2 se muestra la predicción obtenida con el Subset del 3, empleando Euler-Maruyama, variance perserving (VP) con inyección linear schedule.

Se pueden observar que las imágenes representan 3 bastante bien, sin embargo, como era de esperar, hay más ruido y menos precisión que usando PF-ODE.También cabe resaltar que los resultados son ligeramente mejores que los obtenidos anteriormente con EM-VE, esto al igual que con el caso para PF-ODE, se debe a que la varianza de VP está acotada y permite una eliminación de ruido más controlada y precisa. A continuación, se muestran los resultados obtenidos con el dataset completo.

En la apéndice A.12.2 se muestra la predicción obtenida con el dataset completo, empleando EM, variance perserving (VP) con inyección linear schedule.

Los resultados mostrados presentan una calidad aceptable, sin embargo, parecen ser peores que los obtenidos con VE. Esto pruede provocar confusión ya que se contradice con los resultados comentados para un dígito; sin embargo, esto se debe a que VP-linear al tener la varianza acotada puede limitar la exploración entre modos, haciendo que el modelo se quede cerca de los modos más frecuentes y descuidar otros (se ve muy bien este

efecto en el número 4), por el contrario, VE no tiene varianza acotada lo que le permite explorar mejor el espacio entre modos, cubriendo mejor los 10 dígitos y aumentando la diversidad.

Por último se han generado imágenes a color para este sampler y noise schedule:

En la apéndice A.12.2 se muestra la predicción a color obtenida con el dataset completo, empleando EM, variance perserving (VP) con inyección linear schedule.

Al igual que pasa con VE (Variance Exploding), la resolución disminuye, se presentan 'manchas' cromáticas y pérdida de precisión (se observa que los números no están tan bien definidos que con una escala de grises). De la misma manera, una solución sería aumentando el número de épocas o usando una inyección de ruido del coseno, sin embargo, este último método no será válido para este sampler (Euler-Maruyama). El BPD presentado tiene media = 5.4474 y desviación = 1.2526.

Variance Perserving (VP) con inyección coseno (*Cosine Schedule*)

Se esperan unas imágenes de muy mala calidad o un error de generación. Debido a que Cosine no es compatible con Euler-Maruyama porque en el instante incial t=0 la señal residual $\bar{\alpha}_t = 1$, lo que provoca que términos como $\frac{1}{\sqrt{1-\bar{\alpha}_t}}$ diverjan a infinito y generen inestabilidades numéricas. (Justificación numérica en *Apéndice, Material técnico, VP Cosine schedule*)

En la apéndice A.12.2 se muestra la predicción obtenida con el Subset del 3, empleando Euler-Maruyama, variance perserving (VP) con inyección cosine schedule.

Como se puede observar, los valores obtenidos presentan mucho ruido además de no presentar formas bien definidas, como era de esperar. Dado que ya se ha demostrado su incompatibilidad con un Subset pequeño, no se entrenará con el dataset completo.

Variance Perserving (VP) con $\beta = 1$

Se esperan resultados malos en comparación con VE, debido al problema indicado arriba. Y peores resultados en comparación con VP linear.

En la apéndice A.12.2 se muestra la predicción obtenida con el Subset del 3, empleando Euler-Maruyama, variance perserving (VP) con β constante.

Como era de esperar, el resultado es bastante malo, no se representan bien los dígitos y hay mucho ruido, esto es debido a que con $beta$ constante se inyecta ruido de manera uniforme y no funciona bien con Euler-Maruyama porque el integrador estocástico introduce fluctuaciones aleatorias en cada paso. Al mantener β fija, la mangnitud de ruido añadido es simpre la misma, lo que provoca que en los primeros pasos se destruya información relevante (detalles), mientras que en los últimos pasos se siguen inyectando perturbaciones que impiden la correcta convergencia. Sin embargo, y como se ha podido observar, con linear schedule, se inyecta de forma creciente lo que permite mejor refinamiento de detalles, obteniendo mejor calidad. Una vez más, no se probará con el dataset completo, ya que los resultados esperados serán muy baja calidad, viendo mayoritariamente ruido.

5

Conclusiones

En la siguiente sección se comentan las conclusiones del problema abordado.

Tras las diferentes pruebas con los samplers 'Euler-Maruyama' y 'PF-ODE', observamos que todos los resultados obtenidos usando el método de EM presentan peor calidad que PF-ODE. Esto se debe a que EM añade ruido estocástico en cada paso de integración, lo que introduce incertidumbre y requiere un mayor número de pasos (>1000) y pasos más pequeños para obtener resultados de calidad aceptable; sin embargo, con la suficiente capacidad computacional, EM nos devolverá valores de mayor calidad, gracias al uso de Predictor-Corrector, que permite explorar más caminos y converger a modos más definidos de la distribución, evitando la deriva inducida por el ruido. En cambio PF-ODE, al eliminar el término estocástico, converge más rápidamente y con menos pasos (200-800), produciendo imágenes más nítidas y estables, tal y como representan los resultados obtenidos.

Por otra parte, en el caso de EM no es posible usar schedules como el coseno, ya que no llega a convergir. Con EM, los mejores resultados se obtuvieron con VE (variance exploding), mientras que VP con β constante devolvió resultados muy deficientes, y VP con schedule lineal mejoró ligeramente, sin alcanzar el nivel de VE. Para PF-ODE, se observó un comportamiento distinto. Con el dataset compelto de MNIST, VE ofreció los mejores resultados, debido a su mayor flexibilidad para explorar distribuciones multimodales. Sin embargo, para el caso de un solo dígito, los resultados más nítidos se lograron con VP y linear schedule. Tanto el schedule lineal como el coseno con ODE proporcionarion resultados sobresalientes, mientras que con VP β constante volvió a bajar la calidad, ya que inyecta ruido demasiado rápido al principio del proceso, destruyendo información relevante antes de que el modelo pueda aprender.

Las métricas de BPD (bits per dimension) obtenidas reflejan el comportamiento descrito. Para el caso de un solo dígito, el schedule lineal superó al coseno, probablemente debido al número reducido de épocas de entrenamiento (30), siendo este insuficiente para que el coseno mostrase su capacidad. Con más épocas, se espera que el coseno iguale o supere al lineal.

Por último, para obtener los mejores resultados en el dataset completo de MNIST, se recomienda utilizar el proceso VP (Variance Preserving) con schedule coseno y el sampler PF-ODE, entrenando con al menos 200 épocas y un número de pasos de muestreo de 500 a 1000. Esta configuración permite aprovechar la varianza acotada de VP para una eliminación precisa del ruido, la suavidad del coseno para preservar detalles finos y la eficiencia de ODE para una generación rápida y estable. Si lo que se busca es más diversidad y mejor calidad, y disponemos de los recursos computacionales necesarios, será mejor Euler Maruyama.

Bibliografía

[1] E. Mind, "Variance-exploding sde analysis." https://www.emergentmind.com/topics/variance-exploding-stochastic-differential-equation, July 2025.

[2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," *arXiv preprint arXiv:2011.13456*, 2021.

[3] Wikipedia, "Ecuación diferencial estocástica." https://es.wikipedia.org/wiki/Ecuación_diferencial_estocástica, 2024.

[4] Wikipedia, "Movimiento browniano." https://es.wikipedia.org/wiki/Movimiento_browniano, 2024.

[5] E. Mind, "Variance-preserving diffusion process." https://www.emergentmind.com/topics/variance-preserving-diffusion-process, Jan. 2026.

[6] Milvus, "What techniques are available to accelerate the sampling process." https://milvus.io/ai-quick-reference/what-techniques-are-available-to-accelerate-the-sampling-process.

[7] IBM, "Diffusion models." https://www.ibm.com/es-es/think/topics/diffusion-models.

[8] Wikipedia, "U-net." https://es.wikipedia.org/wiki/U-Net, 2024.

[9] Wikipedia, "Random feature." https://en.wikipedia.org/wiki/Random_feature, 2024.

[10] G. Gundersen, "Random fourier features." https://gregorygundersen.com/blog/2019/12/23/random-fourier-features/, Dec. 2019.

[11] Wikipedia, "Predictor–corrector method." https://en.wikipedia.org/wiki/Predictor\T1\textendashcorrector_method, 2024.

[12] E. Mind, "Probability flow ordinary differential equation (pf ode)." https://www.emergentmind.com/topics/probability-flow-ordinary-differential-equation-pf-ode.

[13] Mapengsen, "Probability flow ode." https://mapengsen.blog.csdn.net/article/details/138056790, May 2024.

[14] Wikipedia, "Distancia de inicio de fréchet." https://es.wikipedia.org/wiki/Distancia_de_inicio_de_Fréchet, 2024.

[15] Wikipedia, "Inception score." https://en.wikipedia.org/wiki/Inception_score, 2024.

[16] YData, "Conditional sampling for synthetic data generation." https://docs.sdk.ydata.ai/latest/synthetic_data/conditional_sampling/.

[17] Wikipedia, "Imputation (statistics)." https://en.wikipedia.org/wiki/Imputation_(statistics), 2024.

[18] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. V. Gool, "Repaint: Inpainting using denoising diffusion probabilistic models," *arXiv preprint arXiv:2201.09865*, 2022.

[19] Wikipedia, "Ecuación de fokker-planck." https://en.wikipedia.org/wiki/Fokker\T1\textendashPlanck_equation, 2024.

[20] A. Topics, "Diffusion models." https://aitopics.org/doc/conferences:D16296E2.

Apéndices

A

Material técnico (derivaciones y fotos)

En esta sección se incluyen derivaciones matemáticas, ilustraciones de resultados y otro material complementario que, por razones de espacio o para no interrumpir la fluidez de la lectura, no fue posible incorporar en el cuerpo principal del trabajo.

A.1 Fokker-Planck. Ecuación completa

$$\frac{\partial}{\partial t}p(x,t) = -\frac{\partial}{\partial x}\left[\mu(x,t)\,p(x,t)\right] + \frac{\partial^2}{\partial x^2}\left[D(x,t)\,p(x,t)\right]$$

A.2 Movimiento Browniano

El movimiento browniano (o proceso de Wiener) es el proceso estocástico más simple, definido por dw con media $\mu = 0$ y varianza $\sigma^2 = 2Dt$. El proceso VE se puede entender como un movimiento browniano con coeficiente de difusión dependiente del tiempo $\sigma(t)$

A.3 Variance Exploding (VE)/Brownian Motion. Derivación de g(t)

Resolución de la SDE. Integramos desde 0 hasta t: $\int_0^t dx_s = \int_0^t g(s)dw_s$, $x_t - x_0 = \int_0^t g(s)dw_s$.

Dado que la integral estocástica de una función determinista es una variable aleatoria gaussiana con media cero y varianza $\int_0^t g(s)^2 ds$, podemos escribir: $x_t - x_0 = \sigma N(0,1)$, como $w_0 = 0$:$x_t = x_0 + \sigma \cdot w_t$, donde $\sigma^2(t) = \int_0^t g(s)^2 ds$, por tanto, obtenemos que: $g(t) = \sqrt{\frac{d\sigma^2(t)}{dt}}$

Lo que implica que la evolución de la variable x(t) está gobernada únicamente por la difusión, es decir, no existe ninguna fuerza determinista que empuje la variable hacia un valor específico; su movimiento se debe exclusivamente a las fluctuaciones aleatorias del ruido blanco escalado por g(t).

Para que la varianza 'explote' desde un valor mínimo σ_{min} hasta un valor máximo σ_{max} se elige: $\sigma(t) = \sigma_{min}(\frac{\sigma_{max}}{\sigma_{min}})^2$, lo que garantiza que en $t = 0 : \sigma(0) = \sigma_{min}$ y en $t = 1 : \sigma(1) = \sigma_{max}$

Para obtener el valor $g(t)$, sustituimos sigma en la ecuación y realizamos la derivada.

$$\sigma^2(t) = \sigma_{min}^2(\frac{\sigma_{max}}{\sigma_{min}})^{2t};\ \frac{d\sigma^2(t)}{dt} = \sigma_{min}^2(\frac{\sigma_{max}}{\sigma_{min}})^{2t} \cdot 2\ln\frac{\sigma_{max}}{\sigma_{min}} = \sigma^2(t) \cdot 2\ln\frac{\sigma_{max}}{\sigma_{min}}.$$

$$g(t) = \sigma(t)\sqrt{2\ln\frac{\sigma_{max}}{\sigma_{min}}}$$

A.4 Variance Preserving (VP) / Ornstein-Unlenbeck. Distribución condicional

Distribución condicional

$$p(x_t|x_0) = N(\mu(t) \cdot x_0, \sigma^2(t) \cdot I)$$

Por las propiedades fundamentales del movimiento browniano, sabemos que:

- Media de w_t = 0
- Varianza de w_t = t
- Desviación típica de w_t = $\sqrt{t}$

Teniendo en cuenta las características superiores, calculamos la media y varianza de x_t

- $E[x_t|x_0] = E[x_0e^{-t/2} + e^{-t/2}\int_0^t e^{s/2}|x_0] = x_0e^{-t/2} + e^{-t/2}\int_0^t e^{s/2}$, dado que la intregal estocástica tiene media = 0.
- $V[x_t|x_0] = Var[x_0e^{-t/2} + e^{-t/2}\int_0^t e^{s/2}|x_0] = x_0e^{-t/2} + e^{-t/2}\int_0^t e^{s/2}$, dado que $Var[\int_0^t e^{s/2}dw_s] = \int_0^t (e^{s/2})^2 ds = \int_0^t e^s ds = e^t - 1$ (isometría de Itô)

A.5 Comparativa entre VE y VP

- **Coeficiente de deriva:** En VE es nulo, mientras que en VP viene dada por $-\frac{1}{2}\mathbf{x}\,dt$, lo que introduce una fuerza que empuja la variable hacia cero
- **Evolución de la varianza**: En VE la varianza crece indefinidamente (explota) con el tiempo, mientras que en VP permanece estable, tendiendo a 1 cuando $t \to \infty$
- **Distribución final**: En VE obtenemos una normal con varianza muy grande, mientras que en VP, una normal estándar (varianza = 1)
- **Estabilidad numérica**: VP es más estable debido a que su varianza está acotada
- **Complejidad**: VE es más simple de implementar al no tener término de deriva, pero VP ofrece mejores propiedades de convergencia

A.6 Implicaciones de U-NET

Aunque la U-Net sea muy potente, cuando la imagen de entrada tiene mucho ruido, es prácticamente imposible que la red recupere una imagen limpia en un solo paso, ya que existen infinitas imágenes posibles compatibles con ese ruido. Si forzamos una sola pasada, el resultado sería borroso y con mucho ruido residual. Por eso, en los diffusion models se utiliza un proceso iterativo: comenzamos con ruido puro y aplicamos la U-Net muchas veces (por ejemplo, 1000 pasos), donde cada vez la imagen de entrada tiene un poco menos de ruido que la anterior. De esta forma, en lugar de pedirle algo mágico (la imagen 'buena' en una sola pasada), le pedimos muchos pequeños refinamientos sucesivos, obteniendo finalmente imágenes nítidas y de alta calidad.

A.7 Comparativa entre samplers

- **Euler-Maruyama**: Es el método más simple, basado en una discretización directa de la SDE reversa. Cada paso es muy rápido, pero requiere un número elevado de pasos (500-2000) para obtener resultados de calidad. Su implementación es sencilla y se obtiene una calidad de imagen relativamente buena.
- **Predictor-Corrector**: Combina un paso predictor (Euler-Maruyama) con múltiples pasos correctores (Langevin dynamics). Aunque cada paso es más costoso, permite reducir el número de iteraciones y produce imágenes de mayor calidad. Es la opción preferida cuando se prioriza la fidelidad. Obtiene una calidaad de imagen muy buena.
- **Probability Flow ODE**: Convierte el proceso estcocástico en una EDO determinista eliminando el ruido. Es más rápido y estable, pero al ser determinista, la diversidad de loas muestras generadas pueden ser menor.

A.8 Construcción de Predictor-corrector

Para construir un método predictor-corrector simple a partir del método de Euler y la regla del trapecio, realizamos lo siguiente:

- Ecuación diferencial ordinaria (EDO): $\frac{dy}{dt} = f(t, y),\ y(t_0) = y_0$
- Paso 1. Predicción: partiendo del valor actual y_i, calculamos un valor de estimación inicial $\tilde{y}_{i-1}$ mediante el método de Euler:

$$\tilde{y}_{i+1} = y_i + hf(t_i, y_i)$$

donde h = tamaño del paso

- Paso 2. Corrección: Mejoramos la estimación inicial usando la regla trapezoidal:

$$y_{i+1} = y_i + \frac{1}{2}h\big(f(t_i, y_i) + f(t_{i+1}, \tilde{y}_{i+1})\big)$$

Este valor será el usado para el siguiente paso.

A.9 Comparativa de métricas y escenarios de uso

- **BPD (bits per dimension)**: Mide la eficiencia de comprensión del modelo. Útil para comparar la estimación de densidad, se puede calcular con pocas muestras; sin embargo, no mide directamente la calidad visual (una

imagen borrosa puede tener bien BPD)

- **FID (Fréchet Inception Distance)**: Compara la distribución de las características de las imágenes creadas con la de las reales. Es la métrica usada hoy en día porque correlaciona bien con la percepción humana. Desventaja: necesita muchas imágenes (+10000) para ser fiable
- **IS (Inception Score)**: Mide la calidad y diversidad sin necesidad de imágenes reales. Sin embargo, no detecta si las imágenes generadas se parecen o no a las reales, por lo que se suele complementar con FID

A.10 Impactos de Linear y Cosine

Linear schedules son más fáciles de depurar y más rápidos de entrenar, pero pueden tener menor rendimiento en tareas complejas. Por otro lado, Cosine schedule suele producir mejores resultados, aunque requieren más recursos computacionales y un mayor ajuste de parámetros.

A.11 Derivaciones de mu_t y $sigma_t$

A.11.1 VE (Variance Exploding)

Como se ha visto en la sección de *Estado del Arte*, la SDE que define el proceso es $dx = 0 \cdot dt + \sigma(t)dW$, podemos observar la ausencia del parámetro β, lo que implica que el ruido se controla directamente mediante la función $\sigma(t)$, que determina la desviación típica de la perturbación. La varianza no está acotada y crece indefinidamente, por lo que noise_schedules como el cosine (que está acotado) no se puede implementar. Sí es cierto que linear no está 'acotado' y por tanto sí se podría implementar, pero no con la función obtenida/típica de linear, donde nos devuelve el valor de $beta$ (ya que VE no tiene ese parámetro), sino definiendo $\sigma(t)$

En la implementación realizada para VE, se ha optado por un schedule de raíz cuadrada, definido como $\sigma(t) = \sigma\sqrt{t}$, también serían posibles otras opciones como el crecimiento lineal o exponencial.

Obtención de μ_t y σ_t:

Por las propiedades fundamentales del movimiento browniano, sabemos que:

- Media de w_t = 0
- Varianza de w_t = t
- Desviación típica de w_t = $\sqrt{t}$

Teniendo en cuenta las características superiores, calculamos la media y varianza de x_t

- $E[x_t|x_0] = E[x_0 + \sigma \cdot w_t|x_0] = x_0 + \sigma \cdot E[w_t] = x_0 + \sigma \cdot 0 = x_0.$
- $V[x_t|x_0] = Var[x_0 + \sigma \cdot w_t|x_0] = \sigma^2 \cdot t$

A.11.2 VP (Variance Perserving)

Como se ha visto en la sección de *Estado del Arte*, la SDE que define el proceso es $dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dW$, donde el schedule elegido define la evolución del coeficiente $\beta(t)$, el cual controla la tasa de destrucción de señal y permanece acotado.

En la implementación realizada para VP, se ha probado con linear schedule, cosine schedule y beta constante.

Linear schedule

Linear schedule define la varianza del ruido como una función lineal del tiempo:

$$\beta_t = \beta_{\min} + \frac{t}{T}(\beta_{\max} - \beta_{\min})$$

La obtención de μ_t y σ_t.

- μ_t. Sabemos que la fórmula derivada μ_t de VP es $\mu(t) = x_0 \exp^{-\frac{1}{2}\bar{\beta_t}}$, donde $\bar{\beta(t)} = \beta_t$. Por tanto:

$$\bar{\beta(t)} = \int_0^t \beta(s)ds = \beta_{min}t + \frac{1}{2}\frac{\beta_{max} - \beta_{min}}{T}t^2$$

  Sustituyendo en la ecuación de μ_t, obtenemos:

$$\mu(t) = x_0 \exp^{-\frac{1}{2}\cdot(\beta_{min}t + \frac{1}{2}\frac{\beta_{max}-\beta_{min}}{T}t^2)}$$

- σ_t. De igual manera, sabemos que en VP $\sigma(t) = \sqrt{1 - exp^{-\beta_t}}$. Por tanto:

$$\sigma(t) = \sqrt{1 - exp^{-(\beta_{min}t + \frac{1}{2}\frac{\beta_{max}-\beta_{min}}{T}t^2)}}$$

Cosine schedule

Cosine schedule nos proporciona la expresión de α, la varianza marginal de la señal: $\alpha_t = \cos^2(\frac{t+s}{1+s}\frac{\pi}{2})$. Se sabe que la distribución marginal en tiempo t es: $p(x_t|x_0) = N(x_t; \mu_t, \sigma_t^2 I)$, donde $\mu_t = x_0\sqrt{\alpha_t}$ y $\sigma_t^2 = 1 - \alpha$. Sutitiyendo α_t en las ecuaciones, obtenemos:

$$\mu_t = x_0 \cdot \cos(\frac{t+s}{1+s}\frac{\pi}{2})\sigma_t = 1 - \cos^2(\frac{t+s}{1+s}\frac{\pi}{2}) = \sin^2(\frac{t+s}{1+s}\frac{\pi}{2})$$

También se puede obtener la expresión obteniendo el parámetro de β gracias a la relación $\alpha_t = exp(-\int_0^t \beta(s)ds)$.

Derivación de β para Cosine Schedule

La integral de la tasa de ruido $\beta(t)$ se relaciona con α_t mediante: $\alpha_t = exp(-\int_0^t \beta(s)ds)$, por tanto: $\int_0^t \beta(s)ds = -ln(\alpha_t)$ (por las propiedades del logaritmo).

$$\int_0^t \beta(s)ds = -ln[\cos^2(\frac{t+s}{1+s}\frac{\pi}{2})] = -2ln[\cos(\frac{t+s}{1+s}\frac{\pi}{2})]$$

Por la siguiente propiedad de las integrales, $Si\ F(t) = \int_a^b f(s)ds\ entonces\ \frac{dF}{dt} = f(t)$, obtenemos que:

$$\beta(t) = \frac{d}{dt}[-2ln(\cos(\frac{t+s}{1+s}\frac{\pi}{2}))] = 2 \cdot \frac{1}{\cos(\frac{t+s}{1+s}\frac{\pi}{2})} \cdot \sin(\frac{t+s}{1+s}\frac{\pi}{2}) \cdot \frac{\pi}{2(1+s)} = \pi \cdot \frac{\tan(\frac{t+s}{1+s}\frac{\pi}{2})}{1+s}$$

donde $\frac{d}{dt}\frac{t+s}{1+s}\frac{\pi}{2} = \frac{\pi(2(1+s))}{2^2(1+s)^2} = \frac{\pi}{2(1+s)}$

A.12 Ilustraciones de los resultados

A.12.1 Ilustraciones para PF-ODE

En cada subsección se presentan tres imágenes, una con únicamente el dígito 3 (prueba unitaria), una con el dataset completo (dígitos 0-9) en escala de grises, y una última del dataset completo (dígitos 0-9) en rgb.

Variance Exploding (VE) con inyección raíz exponencial

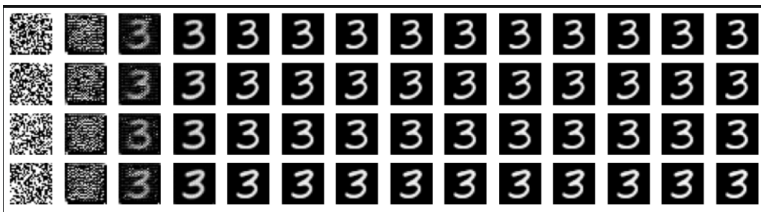


Figura: Predicción del dígito 3 con ODE-VE y exponential schedule

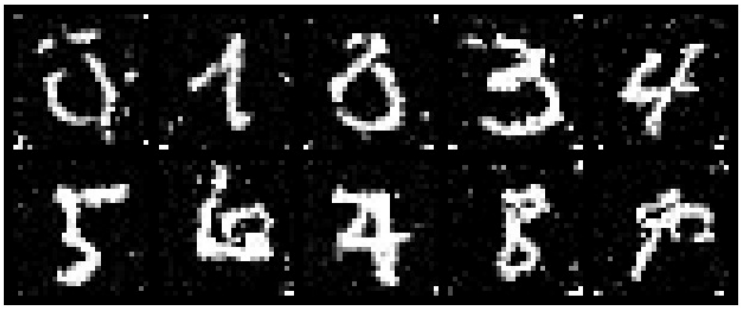


Figura: Predicción del dataset completo con ODE-VE y exponential schedule

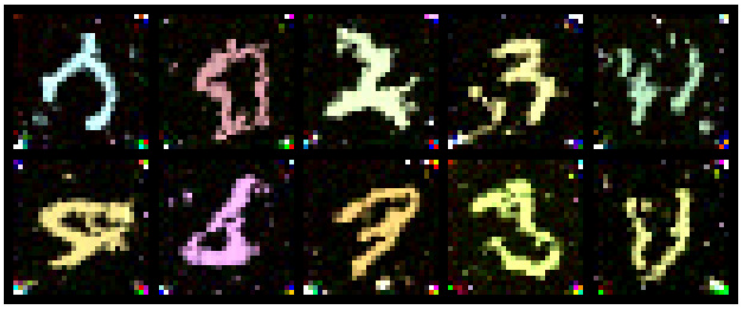


Figura: Predicción del dataset completo a color con ODE-VE y exponential schedule

Variance Exploding (VE) con inyección raíz cuadrada

Se muestra el resultado obtenido con el Subset del dígito 3:

En la apéndice A.12.1 se muestra la predicción obtenida con un Subset únicamente del dígito 3, empleando PF-ODE, variance exploding con inyección squared schedule



Figura: Predicción del dígito 3 con ODE-VE y squared schedule

Como se ha podido comprobar, con un Subset pequeño se obtienen resultados favorables, el 3 se ve de manera clara y si bien presenta algo de ruido, este es muy reducido. Esto indica que el modelo es capaz de aprender la estructura general del dígito.

Seguidamente, se procede al entrenamiento del modelo con todo el dataset. El resultado esperado es una ilustración en la que se muestran los valores de todos los números (0-9) donde se verán bien identificados aunque presentando cierta borrosidad en los bordes debido a la naturaleza del proceso VE (Variance Exploding), en el que la varianza del ruido no está acotada y crece indefinidamente, lo que dificulta la eliminación precisa del ruido en los detalles de alta frecuencia, como son los bordes del dígito.

En la apéndice A.12.1 se muestra la predicción obtenida con el dataset completo (0-9), empleando PF-ODE, variance exploding con inyección squared schedule

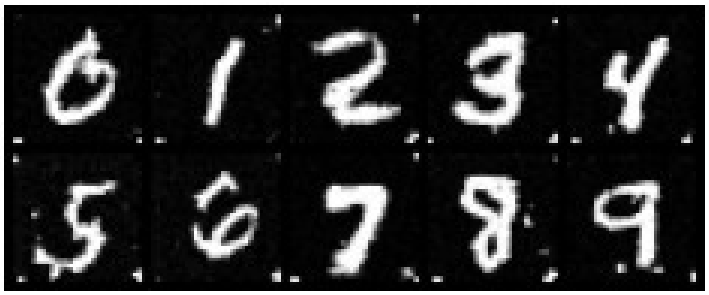


Figura: Predicción de dígitos 0-9 con ODE-VE y squared schedule

Los resultados a color no serán presentados.

Variance Perserving (VP) con inyección linear schedule



Figura: Predicción del dígito 3 con ODE-VP y linear schedule

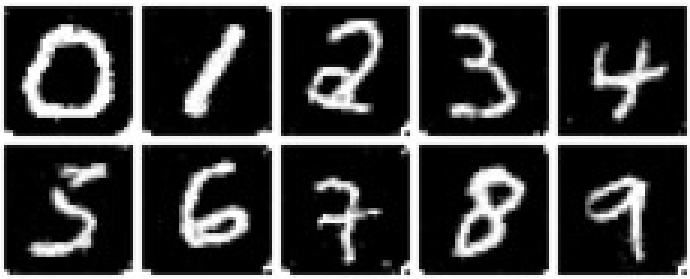


Figura: Predicción de dígitos 0-9 con ODE-VP y linear schedule

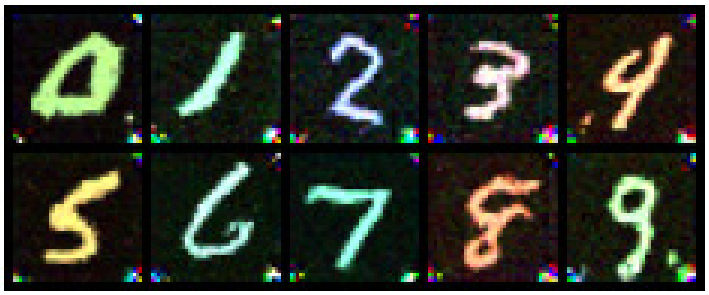


Figura: Predicción de dígitos 0-9 a color con ODE-VP y linear schedule

Variance Perserving (VP) con inyección coseno (*Cosine Schedule*)

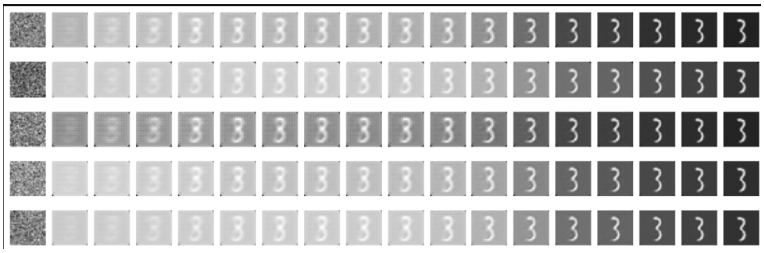


Figura: Predicción del dígito 3 con ODE-VP y cosine schedule

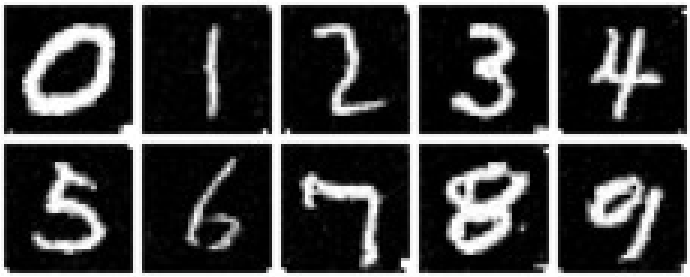


Figura: Predicción de dígitos 0-9 con ODE-VP y cosine schedule



Figura: Predicción de dígitos 0-9 a color con ODE-VP y cosine schedule

Variance Perserving (VP) con $\beta = 1$

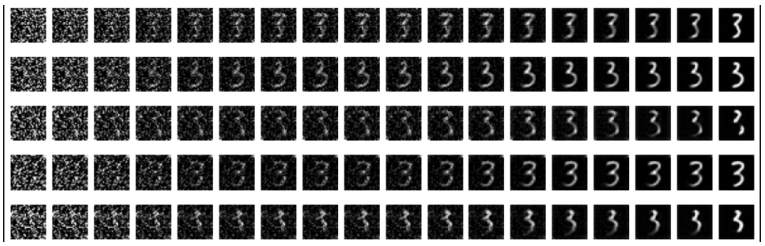


Figura: Predicción del dígito 3 con ODE-VP y β constante

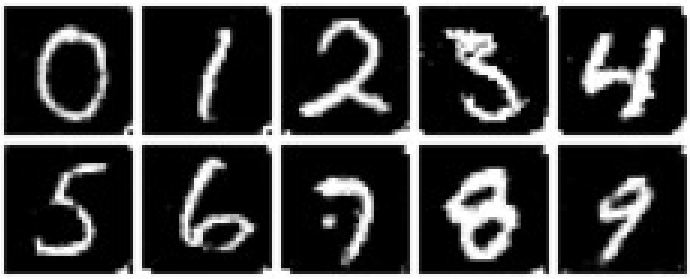


Figura: Predicción de dígitos 0-9 con ODE-VP y β constante



Figura: Predicción de dígitos 0-9 a color con ODE-VP y β constante

A.12.2 Ilustraciones para EULER-MARUYAMA

En cada subsección se presentan dos imágenes, una con únicamente el dígito 3 (prueba unitaria), una con el dataset completo (dígitos 0-9) en escala de grises, y adicionalmente una última del dataset completo (dígitos 0-9) en rgb.

Variance Exploding (VE) con inyección exponencial

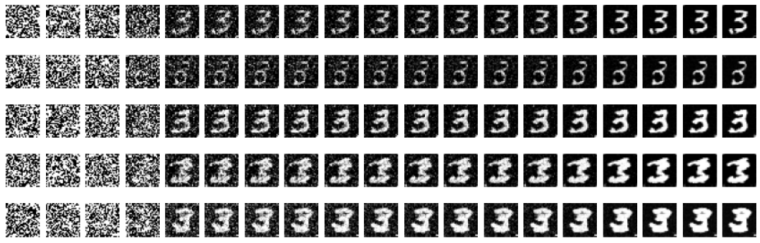


Figura: Predicción del dígito 3 con EM-VE y ruido exponencial

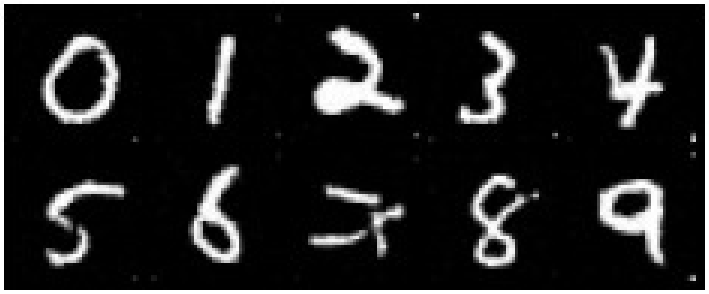


Figura: Predicción del dataset completo con EM-VE y ruido exponencial

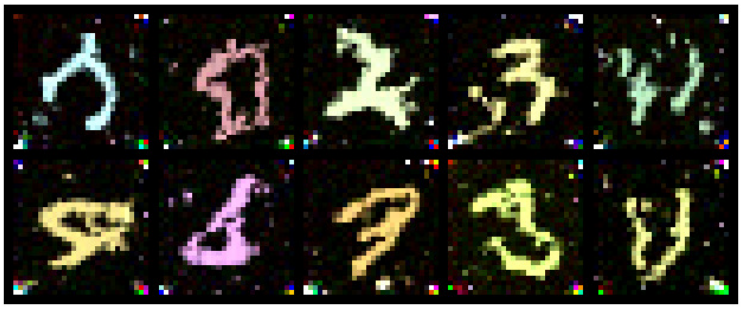


Figura: Predicción del dataset completo a color con EM-VE y ruido exponencial

Variance Exploding (VE) con inyección raíz cuadrada

Destacamos el aumento tanto del número de épocas como del tamaño de pasos con respecto a los modelos de Subsets empleados con ODE. Esto es debido al problema indicado anteriormente (Euler-Maruyama introduce ruido estocástico en cada paso de integración), por tanto, para obtener unos resultados moderadamente satisfactorios se han de aumentar esos parámetros, permitiendo que, al reducir el tamaño de cada paso y aumentar los pasos totales, lo que implica aumentar el número de iteraciones para recorrer en el mismo intervalo temporal, se pueda aproximar con mayor precisión la trayectoria estocástica y corregir las desviaciones inducidas por el ruido en cada integración.

En la apéndice A.12.2 se muestra la predicción obtenida con el Subset del 3, empleando Euler-Maruyama, variance exploding (VE) con inyección squared schedule.

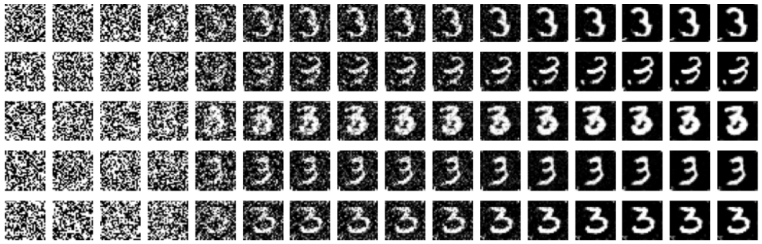


Figura: Predicción del dígito 3 con EM-VE y squared schedule

Como se puede observar, se han podido los 3 de manera medianamente buena, pero con mucho ruido y con bordes poco definidos. A continuación, se muestra el resultado entrenado con el dataset completo, cuya solución debería mostrar números reconcocibles pero de bastante peor calidad que los entrenados con PF-ODE asumiendo mismo número de pasos y épocas.

En la apéndice A.12.2 se muestra la predicción obtenida con el dataset completo, empleando EM, variance exploding (VE) con squeared schedule.

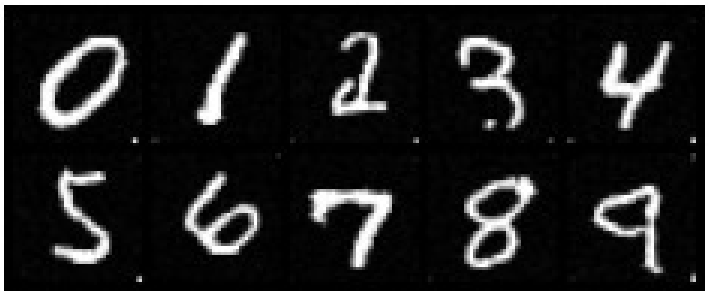


Figura: Predicción de dígitos 0-9 con EM-VE y squared schedule

Dada la imagen superior, vemos que el resultado es ligeramente mejor que el obtenido con PF-ODE, puede parecer contradictorio a lo indicado anteriormente; sin embargo, este resultado ha sido entrenado con 100 épocas y 2000 pasos (1200 pasos más que con PF-ODE). Por tanto, la mejora observada no se debe a una superioridad intrínseca del sampler EM sobre PF-ODE, sino al mayor esfuerzo computacional invertido. De hecho, como se ha discutido anteriormente, PF-ODE es teóricamente más eficiente por paso, pero su rendimiento final depende críticamente de estos hiperparámetros.

Por último se muestra la imagen generada a color para este sampler, el color de cada número ha sido escogido de manera aleatoria. Los valores de BPD presentados son: media = 5.8592 y desviación = 1.2135:

En la apéndice A.12.2 se muestra la predicción obtenida a color con el dataset completo, empleando EM, variance exploding (VE) con squeared schedule.



Figura: Predicción de dígitos 0-9 a color con EM-VE y squared schedule

Como es de esperar, al añadir color el resultado obtenido presenta peor calidad que el original (blanco y negro), esto se debe al paso de 1 canal a 3 canales, aumentando así la complejidad del problema de generación, dando lugar a mayor ruido y presencia de 'manchas' o imperfecciones cromáticas, pérdida de precisión (el 4 y 9 no se diferencian apenas), y una calidad inferior que en escala de grises. Para mejorar la calidad, se recomienda aumentar el número de épocas de entrenamiento.

Variance Perserving (VP) con inyección linear (*Linear Schedule*)

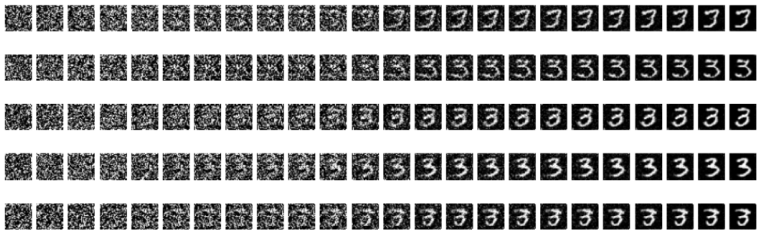


Figura: Predicción del dígito 3 con EM-VP y linear schedule

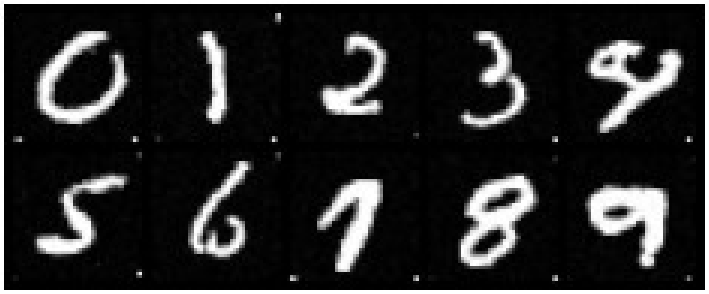


Figura: Predicción de dígitos 0-9 a color con EM y linear schedule



Figura: Predicción de dígitos 0-9 con EM y linear schedule

Variance Perserving (VP) con inyección coseno (*Cosine Schedule*)

La señal residual de Cosine se define como: $\bar{\alpha_t} = \frac{f(t)}{f(0)}$, donde

- $f(t) = \cos^2(\frac{t+s}{1+s}\frac{\pi}{2})$
- $f(0) = \cos^2(\frac{s}{1+s}\frac{\pi}{2})$

En el instante t=0, se cumple que $\bar{\alpha_0} = 1$. Durantte el muestreo reverso con EM es necesario calcular términso como $\frac{1}{\sqrt{1-\bar{\alpha_t}}}$. Por tanto, cuando t -> 0, tenemos que $\bar{\alpha_0} -> 1$, por tanto $1-\bar{\alpha_t} -> 0$ y consecuentemente:

$$\lim_{t->0} \frac{1}{\sqrt{1-\bar{\alpha_t}}} -> \infty$$

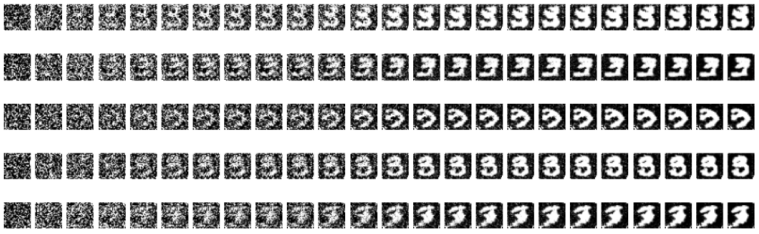


Figura: Predicción del dígito 3 con EM-VP y cosine schedule

Variance Perserving (VP) con $\beta = 1$

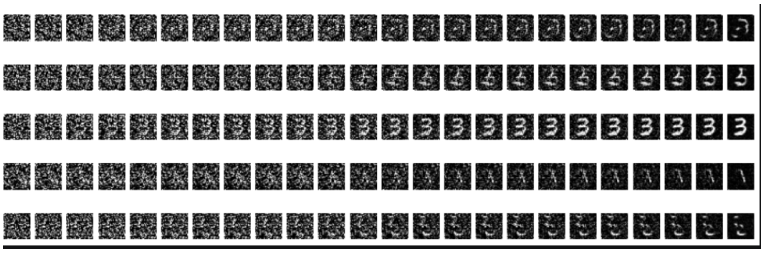


Figura: Predicción del dígito 3 con EM-VP y β constante

B

Anexo

B.1 Aproximación de Hutchison

Técnica de estimación estocástica que permite calcular la traza de una matriz sin necesidad de construirla explícitamente. Dada por:

$$\mathrm{Tr}(A) = \mathbb{E}_{\boldsymbol{\varepsilon}\sim\mathcal{N}(0,I)}\left[\boldsymbol{\varepsilon}^T A\boldsymbol{\varepsilon}\right]$$

donde A es una matriz cuadrada y ε es un vector aleatorio con distribución normal estándar.

B.2 Square root schedule

Método que define cómo añadir ruido al modelo. A diferencia de $LinearSchedule$ o $CosineSchedule$, Square Root Schedule añade ruido rápidamente en los primeros pasos y luego ralentiza su crecimiento. Lo que evita que el modelo pase demasiados pasos en niveles de ruido muy altos, que pueden ser difíciles de resolver. Su fórmula viene dada por:

$$\bar{\alpha}_t = 1 - \sqrt{\frac{t}{T} + s}$$

donde:

- t es el paso actual (0-T)
- T es el número total de pasos
- s es el offset (típicamenete 0.0001)


UÁM
Universidad Autónoma
de Madrid